BOB-095 â€” Go-side SSE-broker stress + chaos automation evidence

Revision: 1 **Last modified:** 2026-08-19T00:00:00Z

Scope

`qBitTorrent-go/internal/service/sse_broker_stress_chaos_test.go` â€” Â§11.4.85 stress + chaos coverage for `SSEBroker` (`internal/service/sse_broker.go`) under `go test -race`. Go-side sibling to BOB-094â€™s Python tracker-fetch coverage.

Scenarios delivered (6 tests, all GREEN under -race)

STRESS

#	Test	Category
1	`TestSSEBrokerStressChaos_SustainedSubscribeUnsubscribe`	sustained-load â€” 1000 cycles + goroutine-leak guard
2	`TestSSEBrokerStressChaos_ConcurrentSubscribers`	concurrent â€” 100 subs Ã— 5 msgs = 500 delivered, no deadlock
3	`TestSSEBrokerStressChaos_BoundaryDoubleUnsubscribe`	boundary â€” double-close idempotency

CHAOS

#	Test	Category
4	`TestSSEBrokerStressChaos_SlowSubscriberDoesNotBlock`	backpressure isolation â€” `default:` clause proof
5	`TestSSEBrokerStressChaos_UnsubscribeDuringPublish`	RLock/Lock ordering

#	Test	Category
6	TestSSEBrokerStressChaos_ProducerFloodBoundedByBuffer	under contention producer-flood bounded by 10-msg buffer

Â§11.4.115 RED-first captures

Natural RED â€” Â§11.4.238 automated-QA-discovers defect

The initial test run of scenario #3 (double-unsubscribe) uncovered a REAL defect the source-side review had missed: `close(ch)` panicked with `close of closed channel`. Captured in `evidence/red_capture_double_unsubscribe.txt`. Fix landed in the same commit: `sse_broker.go` `Subscribe()` gates the `close` on registry membership so a second call is a safe no-op (Â§11.4.253 idempotency).

Synthetic RED â€” backpressure mutation

`sse_broker.go` `Publish()` was TEMPORARILY mutated by removing the `select { â€¦ default: â€¦ }` clause (turning it into a blocking send). Scenario #4 wedged as expected; the `-timeout 10s` deadline triggered and the test FAILed. Captured in `evidence/red_capture_backpressure.txt`. The `default:` clause was restored immediately after capture; the GREEN re-run is in `evidence/green_capture_after_restore.txt`.

Evidence layout

- `latency.json` â€” p50/p95/p99 + goroutine deltas (scenario 1)
- `concurrent.json` â€” fan-out totals (scenario 2)
- `boundary_double_unsubscribe.txt` â€” safe no-op proof (scenario 3)
- `chaos_slow_subscriber.json` â€” publisher-not-wedged proof (scenario 4)
- `chaos_unsubscribe_during_publish.json` â€” race + panic counts (scenario 5)
- `chaos_producer_flood.json` â€” buffer-bounded delivery (scenario 6)
- `red_capture_double_unsubscribe.txt` â€” Â§11.4.115 RED (natural, defect-found)
- `red_capture_backpressure.txt` â€” Â§11.4.115 RED (mutation, restored)
- `green_capture_after_restore.txt` â€” final GREEN run, 6/6 PASS

Discipline citations

- Â§11.4.85 stress + chaos test types

- Â§11.4.115(F) machine-written verdict pairs â€” natural RED / mutation RED
- Â§11.4.161 rootless â€” no root/podman needed
- Â§11.4.14 cleanup on every exit â€” `defer unsub()` on every subscribe
- Â§11.4.3 SKIP-with-reason â€” evidence-dir creation failure, never fake pass
- Â§12/Â§12.6/Â§12.12 host safety â€” `GOMAXPROCS=2 nice -n 19`
- Â§11.4.238 automated QA discovers â€” scenario #3 caught a live defect
- Â§11.4.253 idempotency â€” the double-unsubscribe fix